

Resumo: Design de Software (SWEBOK - Capítulo 3)

O design de software é uma área fundamental da engenharia de software que trata da formulação e organização da estrutura interna de um sistema antes da implementação. Ele organiza os componentes, as interfaces e a arquitetura do software para que ele seja eficiente, fácil de manter e reutilizar. Seu principal objetivo é garantir que o sistema atenda tanto aos requisitos funcionais quanto aos não funcionais.

Alguns princípios importantes são seguidos, como a modularidade, que consiste na divisão do sistema em partes menores, o baixo acoplamento, que trata de evitar dependências desnecessárias entre componentes, e a alta coesão, que garante que cada parte do software tenha um único propósito bem definido.

O processo de design pode ser dividido em três estágios: Architectural Design, que define um modelo computacional, os principais elementos computacionais e os protocolos e relacionamentos entre eles; High-Level Design, que inclui a identificação dos elementos computacionais primários e relacionamentos significativos entre eles; e Detailed Design, que define cada estrutura interna do módulo, focando no detalhamento e na justificativa de escolhas de algoritmos, acesso a dados e representação de dados.

A qualidade do design depende de características como a capacidade do software de executar múltiplas tarefas ao mesmo tempo, organizar o fluxo de controle, lidar com eventos internos e externos, armazenar dados de forma eficiente, prevenir, mitigar e processar erros e garantir segurança contra ataques, violações de políticas e acessos não autorizados. Além disso, um bom design permite que diferentes sistemas trabalhem juntos e que o software possa ser adaptado para diferentes necessidades.

Para registrar o design, utilizam-se diagramas e modelos que ajudam a visualizar a estrutura e o comportamento do software. Diagramas de classe e objeto descrevem as relações entre elementos do sistema, enquanto diagramas de componentes detalham a interconexão dos módulos. Diagramas de implantação mostram a alocação do software no hardware, e diagramas entidade-relacionamento representam a modelagem de dados. Além disso, linguagens de descrição de interfaces são utilizadas para definir os serviços expostos pelos componentes, enquanto gráficos de estrutura representam a organização hierárquica dos módulos.

O comportamento dinâmico do software também pode ser descrito por diferentes notações e linguagens. Diagramas de atividade representam o fluxo de execução e possíveis concorrências, enquanto diagramas de interação mostram a troca de mensagens entre objetos. Diagramas de fluxo de dados descrevem como a informação circula no sistema, e fluxogramas ou tabelas de decisão auxiliam na representação lógica dos processos. Diagramas de estados ilustram as transições de estado do software com base em eventos, enquanto linguagens de especificação formal descrevem com precisão o comportamento dos componentes. O pseudocódigo também pode ser utilizado para representar algoritmos de forma estruturada.

Técnicas como Model-Based Design e padrões como MVC e Microservices, bem como DSLs, auxiliam na criação de um design bem organizado e reutilizável.

Existem diferentes abordagens que podem ser adotadas, como o design estruturado baseado em funções e fluxo de dados, o design orientado a objetos com uso de classes e herança e o design baseado em componentes com módulos reutilizáveis. Além disso, há abordagens que focam na experiência do usuário, na reação a eventos ou no domínio específico do problema.

Para garantir a qualidade do design, ele deve ser revisado e testado. Isso inclui revisões e auditorias para verificar a aderência às boas práticas e requisitos, além da análise de atributos como eficiência, manutenibilidade, confiabilidade e usabilidade. Técnicas de análise e avaliação incluem inspeções, simulações e análise estática do código, além da avaliação quantitativa de atributos como complexidade e desempenho. Também é essencial verificar se o software atende aos padrões e regulamentações.

Softwares são criações minuciosas, e o design deles é um processo crítico que impacta diretamente a qualidade, evolução e sucesso de um sistema. A aplicação de boas práticas e estratégias adequadas é fundamental para a criação de soluções sustentáveis que garantam um software eficiente, robusto e alinhado aos requisitos.